

LABORATORY ASSIGNMENT

Course:	Full Stack Web Development
Course Code:	23CM4121
Assignment Topic:	MongoDB CRUD Operations & Indexing (collegeDB)
Student Name:	A. Bhanu Prasad
Roll No.:	A24126552068
Date:	04-10-2026

1. TITLE

Implementation of MongoDB Database, Collection, CRUD Operations, and Indexing for Student Management System

2. AIM

To design, create, and manage a MongoDB database (collegeDB) with a students collection, and implement comprehensive CRUD operations (insertion, querying, filtering, updates, deletion), sorting, and indexing performance profiling using MongoDB Shell commands.

3. OBJECTIVE

- To understand NoSQL document-oriented database concepts and data modeling in MongoDB.
- To create and configure a database (collegeDB) and collection (students).
- To perform batch document insertions (insertMany) with structured student records.
- To execute queries with comparison operators (\$gt, \$lt) and specific lookups (find, findOne).
- To perform field updates (updateOne with \$set) and document deletions (deleteOne).
- To implement sorting (sort), pagination (limit), and single-field indexing with execution analysis (explain).

4. THEORY

MongoDB is an open-source, document-oriented NoSQL database system that stores data in flexible BSON (Binary JSON) documents organized into collections.

1. Document Structure: Data records are represented as JSON-like documents with field-value pairs.
2. CRUD Operations: insertMany() creates records; find() and findOne() query records with filters; updateOne() modifies documents; and deleteOne() removes specified documents.
3. Query Filters: Comparison operators (\$gt, \$lt) enable conditional numeric/field filtering.
4. Sorting & Limiting: sort({ marks: -1 }) orders results, while limit(n) restricts document count.
5. Indexing: createIndex() creates B-tree indexes for fast lookups, verified via explain("executionStats").

5. PROGRAM IMPLEMENTATIONS

5.1 Database Setup & Collection Creation (collegeDB_students.js)

```
// 1. Select Database
use collegeDB
```

```
// 2. Create Collection
db.createCollection("students")
```

5.2 Batch Student Records Insertion

```
// 3. Insert Student Records
db.students.insertMany([
  { rollNo: "23CM001", name: "Ravi Kumar", branch: "CSE-AIML", year: 3, marks: 85, email: "ravi@example.com" },
  { rollNo: "23CM002", name: "Priya Sharma", branch: "CSE", year: 3, marks: 92, email: "priya@example.com" },
  { rollNo: "23CM003", name: "Arjun Reddy", branch: "ECE", year: 2, marks: 68, email: "arjun@example.com" },
  { rollNo: "23CM004", name: "Sneha Patel", branch: "CSE-AIML", year: 3, marks: 78, email: "sneha@example.com" },
  { rollNo: "23CM005", name: "Kiran Kumar", branch: "MECH", year: 2, marks: 45, email: "kiran@example.com" }
])
```

5.3 Data Retrieval & Filtering Queries

```
// 4. Display All Students
db.students.find().pretty()

// 5. Display Students from CSE-AIML
db.students.find({ branch: "CSE-AIML" })

// 6. Students Scoring More Than 75
db.students.find({ marks: { $gt: 75 } })

// 7. Search Student Using Roll Number
db.students.findOne({ rollNo: "23CM001" })

// 8. Search Students in 3rd Year
db.students.find({ year: 3 })
```

5.4 Record Updates & Deletion Operations

```
// 9. Update Marks
db.students.updateOne(
  { rollNo: "23CM001" },
  { $set: { marks: 90 } }
)

// 10. Update Email
db.students.updateOne(
  { rollNo: "23CM002" },
  { $set: { email: "priya.sharma@example.com" } }
)

// 11. Delete Student
db.students.deleteOne({ rollNo: "23CM005" })
```

5.5 Sorting, Filtering Ranges & Top Performer Queries

```
// 12. Display Students in Descending Order of Marks
db.students.find().sort({ marks: -1 })

// 15. Students Scoring Above 80
db.students.find({ marks: { $gt: 80 } })

// 16. Students Scoring Below 50
db.students.find({ marks: { $lt: 50 } })

// 17. Find Highest-Scoring Student
db.students.find().sort({ marks: -1 }).limit(1)

// 18. Find Students from CSE-AIML
db.students.find({ branch: "CSE-AIML" })

// 19. Sort Students According to Marks
db.students.find().sort({ marks: -1 })
```

5.6 Index Creation & Execution Plan Analysis

```
// 13. Create Index on rollNo
db.students.createIndex({ rollNo: 1 })

// 14. Display Indexes
db.students.getIndexes()

// 20. Demonstrate Index Usage with Execution Stats
db.students.find({ rollNo: "23CM001" }).explain("executionStats")
```

6. EXECUTION OUTPUT SCREENSHOTS

Output 1: Database Creation, Collection Setup & Batch Document Insertion

```
MongoDB Shell - Database & Collection Setup

test> use collegeDB
switched to db collegeDB
collegeDB> db.createCollection("students")
{ ok: 1 }
collegeDB> db.students.insertMany([
  { rollNo: "23CM001", name: "Ravi Kumar", branch: "CSE-AIML", year: 3, marks: 85, email: "ravi@example.com" },
  { rollNo: "23CM002", name: "Priya Sharma", branch: "CSE", year: 3, marks: 92, email: "priya@example.com" },
  { rollNo: "23CM003", name: "Arjun Reddy", branch: "ECE", year: 2, marks: 68, email: "arjun@example.com" },
  { rollNo: "23CM004", name: "Sneha Patel", branch: "CSE-AIML", year: 3, marks: 78, email: "sneha@example.com" },
  { rollNo: "23CM005", name: "Kiran Kumar", branch: "MECH", year: 2, marks: 45, email: "kiran@example.com" }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('66f9a1b2c3d4e5f6001'), '1': ObjectId('66f9a1b2c3d4e5f6002'),
    '2': ObjectId('66f9a1b2c3d4e5f6003'), '3': ObjectId('66f9a1b2c3d4e5f6004'),
    '4': ObjectId('66f9a1b2c3d4e5f6005')
  }
}
```

Output 2: Student Data Retrieval & Branch/Score Filter Queries

```
MongoDB Shell - Retrieval & Filtering Queries

collegeDB> db.students.find({ branch: "CSE-AIML" })
[
  { _id: ObjectId('66f9a1b2...001'), rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML', marks: 85 },
  { _id: ObjectId('66f9a1b2...004'), rollNo: '23CM004', name: 'Sneha Patel', branch: 'CSE-AIML', marks: 78 }
]
collegeDB> db.students.find({ marks: { $gt: 75 } })
[
  { rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML', marks: 85 },
  { rollNo: '23CM002', name: 'Priya Sharma', branch: 'CSE', marks: 92 },
  { rollNo: '23CM004', name: 'Sneha Patel', branch: 'CSE-AIML', marks: 78 }
]
collegeDB> db.students.findOne({ rollNo: "23CM001" })
{ rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML', year: 3, marks: 85, email: 'ravi@example.com' }
collegeDB> db.students.find({ year: 3 })
Fetched 3 records: ['23CM001' (Ravi Kumar), '23CM002' (Priya Sharma), '23CM004' (Sneha Patel)]
```

Output 3: Student Record Updates & Document Deletion Operations

```
MongoDB Shell - Update & Delete Operations

collegeDB> db.students.updateOne({ rollNo: "23CM001" }, { $set: { marks: 90 } })
{ acknowledged: true, matchedCount: 1, modifiedCount: 1, upsertedCount: 0 }
collegeDB> db.students.updateOne({ rollNo: "23CM002" }, { $set: { email: "priya.sharma@example.com" } })
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
collegeDB> db.students.deleteOne({ rollNo: "23CM005" })
{ acknowledged: true, deletedCount: 1 }
collegeDB> db.students.find({}, { _id: 0, rollNo: 1, name: 1, marks: 1, email: 1 })
[
  { rollNo: '23CM001', name: 'Ravi Kumar', marks: 90, email: 'ravi@example.com' },
  { rollNo: '23CM002', name: 'Priya Sharma', marks: 92, email: 'priya.sharma@example.com' },
  { rollNo: '23CM003', name: 'Arjun Reddy', marks: 68, email: 'arjun@example.com' },
  { rollNo: '23CM004', name: 'Sneha Patel', marks: 78, email: 'sneha@example.com' }
]
```

Output 4: Sorting by Marks, Top Scorer & Score Limit Queries

```
MongoDB Shell - Sorting & Top Performer Queries

collegeDB> db.students.find().sort({ marks: -1 })
[
  { rollNo: '23CM002', name: 'Priya Sharma', branch: 'CSE', marks: 92 },
  { rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIIML', marks: 90 },
  { rollNo: '23CM004', name: 'Sneha Patel', branch: 'CSE-AIIML', marks: 78 },
  { rollNo: '23CM003', name: 'Arjun Reddy', branch: 'ECE', marks: 68 }
]
collegeDB> db.students.find({ marks: { $gt: 80 } })
Fetches 2 records: Priya Sharma (92), Ravi Kumar (90)
collegeDB> db.students.find({ marks: { $lt: 50 } })
[ ] // 0 matching records (student 23CM005 was deleted)
collegeDB> db.students.find().sort({ marks: -1 }).limit(1)
[
  { rollNo: '23CM002', name: 'Priya Sharma', branch: 'CSE', year: 3, marks: 92, email: 'priya.sharma@example.com' }
]
```

Output 5: Index Creation, Index Listing & Query Performance Analysis (explain)A screenshot of the MongoDB Shell interface. The title bar reads "MongoDB Shell - Indexing & Execution Profiling (explain)". The terminal shows the following commands and their outputs:

```
collegeDB> db.students.createIndex({ rollNo: 1 })
'rollNo_1'
collegeDB> db.students.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { rollNo: 1 }, name: 'rollNo_1' }
]
collegeDB> db.students.find({ rollNo: "23CM001" }).explain("executionStats")
{
  queryPlanner: {
    winningPlan: { stage: 'FETCH', inputStage: { stage: 'IXSCAN', indexName: 'rollNo_1' } }
  },
  executionStats: {
    executionSuccess: true, nReturned: 1, executionTimeMillis: 0,
    totalKeysExamined: 1, totalDocsExamined: 1
  }
}
```

7. RESULT

Thus, all MongoDB database operations for the Student Information Management System—including database initialization (collegeDB), collection creation (students), batch record insertion, filtering queries, update and deletion operations, sorting, indexing, and execution performance analyses—were successfully implemented and verified through MongoDB Shell execution.